



多校A层冲刺NOIP2024模拟赛06

T1 小 Z 的手套 (gloves)

为了解决这个问题，我们需要找到一个算法来匹配左手手套和右手手套，以最小化丑陋度（即尺码差）的最大值。

20%

$N = M$ ，将左手手套和右手手套排序后，首位逐一匹配即可。

50%

发现瓶颈在于匹配，匹配的前提是不会超过答案，因此可以考虑从小到大枚举丑陋度（即尺码差）的最大值，然后对于每一个左手手套贪心匹配右手手套，这个可以通过排序和二分来加速。

同时发现丑陋度（即尺码差）的最大值一定是左手和右手手套尺码组合出来的值，可能的答案不超过 n^2 种，复杂度 $O(n^3)$ 。

100%

我们发现问题具有二分性，可以使用二分答案优化之前的做法解决这个问题。

首先，我们需要确定丑陋度的可能范围。最小丑陋度显然是0（如果恰好有匹配的尺码），最大丑陋度则是左手手套和右手手套中的最大差值。我们可以使用二分答案来在这个范围内搜索最优解。

在二分查找的每一步中，我们假设一个丑陋度阈值 `mid`，然后尝试找到满足这个阈值的最大匹配数。我们可以对左手手套和右手手套的尺码分别进行排序，然后使用双指针的方法从两端开始匹配，如果两只手套的尺码差小于等于 `mid`，则它们可以配对，然后移动两个指针。最终，我们得到了满足丑陋度阈值 `mid` 的最大匹配数。

如果匹配数等于最大能够配对成功的手套数量（即左手手套和右手手套数量中的最小值），那么 `mid` 可能是一个解，我们尝试缩小搜索范围到左半部分；否则，我们需要增加丑陋度阈值，所以搜索范围移动到右半部分。

复杂度 $O(n \log |L_i|)$ 。

```

#include<bits/stdc++.h>
using namespace std;

const int INF=1e9;
const int maxn=100005;
int n,m,l[maxn],r[maxn],L=0,R=INF;

bool check(int x){
    for (int i=1,j=1;i<=n;i++,j++){
        while (j<=m&& r[j]<l[i]-x)j++;
        if (j>m||r[j]>l[i]+x)
            return 0;
    }
    return 1;
}

int main() {
    freopen("gloves.in", "r", stdin);
    freopen("gloves.out", "w", stdout);
    cin>>n>>m;
    for (int i=1;i<=n;i++)cin>>l[i];
    for (int i=1;i<=m;i++)cin>>r[i];
    if (n>m){
        swap(n,m);
        for (int i=1;i<=n;i++)swap(l[i],r[i]);
        for (int i=n+1;i<=m;i++){
            r[i]=l[i];
            l[i]=0;
        }
    }
    sort(l+1,l+n+1);sort(r+1,r+m+1);
    while (L!=R){
        int mid=(L+R)>>1;
        if (check(mid))
            R=mid;
        else
            L=mid+1;
    }
    cout<<L<<endl;
}

```

考察知识点

贪心 二分答案 二分查找

T2 小 Z 的字符串 (string)

30分做法

考虑暴力搜索，每次交换相邻的一对字符，找出其中所有好的字符串，求出最少的交换次数，可以通过 $|S| \leq 12$ 。

另外30分

考虑 0 字符占了一半以上，分为两种情况。

1. $|S|$ 为偶数，此时一定无解。
2. $|S|$ 为奇数，如果 0 字符个数恰好为 $\frac{|S|+1}{2}$ ，那么 0 字符一定占据所有奇数位置，将它们暴力交换到各自的位置上统计交换数即可，否则一定无解。

100分做法

考虑 dp 。

有一个朴素状态 $f[i][j][k][p][0/1/2]$ 表示前 i 个位置上 0, 1, 2 的个数分别为 j, k, p ，且第 i 位上的数字为 0/1/2 的最小花费，显然， $f[i][j][k][p]$ 是由 $f[i-1][j-1][k][p]$, $f[i-1][j][k-1][p]$ 和 $f[i-1][j][k][p-1]$ 转移过来，因为要求不能有相邻的同一种数字，所以转移时 $f[i]$ 和 $f[i-1]$ 的最后一维 0/1/2 不能相同。

然后就是转移代价了。

首先有一个比较显然的性质，即相同数字的相对位置不会改变。

假设当前的转移是

$$f[i-1][j-1][k][p][1/2] \rightarrow f[i][j][k][p][0]$$

那么第 i 位上填的数字为 0，该 0 显然由原序列上第 j 个 0 移动过来，我们可以计算出当前该 0 的位置 pos ，花费代价即为 $pos - i$ 。

k, p 的转移类似。但是现在这个复杂度是 $O(n^4)$ 的，需要优化。

考虑优化，因为 $j + k + p = i$ ，我们可以压缩一维，变成 $f[i][j][k][0/1/2]$ ，表示前 i 个位

置 0, 1, 2 的个数分别为 $j, k, i - j - k$, 第 i 位为 0/1/2 的最小代价, 转移类似。

「代码实现的解释」

解释为什么要最后要除以 2。

因为考虑 12 变到 21, 会发现 $i - pos_i$ 的绝对值是 2, 就是可以理解为每当一个数往前动了一格, 就会有另一个被换的数往后动了一格, 即有一个 $i > pos_i$ 的数绝对值减了 1, 有一个 $i < pos_i$ 的数绝对值减了 1, 你不可能换两个 $i > pos_i$ 或 2 个 $i < pos_i$ 的数, 这是不优的, 因为总能找了 2 个相邻的数, 一个 $i > pos_i$, 一个 $i < pos_i$ 把他们交换掉。

```
#include<bits/stdc++.h>
using namespace std;
int t[3][405], n, c[3], dp[205][205][205][3];
char s[405];
int main () {
    freopen("string.in", "r", stdin);
    freopen("string.out", "w", stdout);
    scanf("%s", s+1);
    int len = strlen(s+1);
    for(int i = 1; s[i]; ++i) t[s[i]-'0'][++c[s[i]-'0']] = i;
    if(c[0]>(len+1)/2 || c[1]>(len+1)/2 || c[2]>(len+1)/2) puts("-1"), exit(0);
    memset(dp, 0x3f, sizeof dp);
    dp[0][0][0][0] = dp[0][0][0][1] = dp[0][0][0][2] = 0;
    for(int i = 0; i <= c[0]; ++i)
        for(int j = 0; j <= c[1]; ++j)
            for(int k = 0; k <= c[2]; ++k) {
                int p = i+j+k;
                if(i) dp[i][j][k][0] = min(dp[i-1][j][k][1], dp[i-1][j][k][2]) + abs(p - t[0][i]);
                if(j) dp[i][j][k][1] = min(dp[i][j-1][k][0], dp[i][j-1][k][2]) + abs(p - t[1][j]);
                if(k) dp[i][j][k][2] = min(dp[i][j][k-1][0], dp[i][j][k-1][1]) + abs(p - t[2][k]);
            }
    printf("%d", min({dp[c[0]][c[1]][c[2]][0], dp[c[0]][c[1]][c[2]][1], dp[c[0]][c[1]][c[2]][2]}));
    return 0;
}
```

T3 一个真实的故事 (truth)

20%

修改可以直接改，对于每一次询问 $O(n^2)$ 枚举左端点判断最早合法的右端点，枚举区间 $[i, j]$ 可以在 $[i, j - 1]$ 基础上判断是否合法求出答案，复杂度 $O(mn^2)$

50%

发现对于每一次询问 $O(n)$ 双指针可以求出答案，复杂度 $O(mn)$

70%

发现只有三个数字，可以尝试记录左右两侧第一个1, 2, 3的位置

发现这样修改太多了，尝试分块均衡，记录每一个块内123，最左和最右的位置，每次扫块内和块间查询和修改即可，每次修改完维护全局的答案，只有块内和块间的答案更新了，更新会控制在 $O(\min(B, \frac{N}{B})) = O(\sqrt{n})$ 。

复杂度 $O(km\sqrt{n})$

70~100%

使用线段树解决，主要考察线段树的 pushup 操作。

可以发现当数字较多时可以考虑合并两块区间的答案：

- 拿左区间拥有的数字中靠右些的，右区间拿靠左的。
- 拿左区间的答案
- 拿右区间的答案

当左右两个块需要合并时，取出左块最右的1-k，取出右块最左的1-k，合并起来按数字大小归并后（也可以直接sort，会多一个log）， $O(k)$ 双指针即可求出合并后的答案，同时我们可以很方便的维护出合并后块的最左的1-k和最右的1-k。

然后会发现好像类似线段树或者分块的数据结构都可以比较快的支持这个操作，复杂度 $O(km \log n)$ (如果使用sort为 $O(km \log n \log k)$)

```

#include <bits/stdc++.h>
#define ll long long
#define pii pair<int, int>
using namespace std;

const int N = 5e4+5, K = 31;

int a[N], n, k, m;

//维护线段树
struct node {
    int l, r;
    int ans, lt[K], rt[K], lst;
} tr[N<<2];

void push_up(int u) {
    //把两边的东西拿出来，排序+双指针
    int res = N, cnt[k+1], have = 0;
    memset(cnt, 0, sizeof(cnt));
    pii temp[2*k+1];
    for (int i=1; i<=k; ++i) temp[i] = {tr[u<<1].rt[i], i};
    for (int i=1; i<=k; ++i) temp[k+i] = {tr[u<<1|1].lt[i], i};
    //排序
    sort(temp+1, temp+2*k+1, [](pii &x, pii &y) {return x.first<y.first;});
    //双指针
    for (int i=1, j=1; i<=2*k; ++i) {
        if (!temp[i].first) continue;
        while (j<i) ++j;
        while (j<=2*k && have<k) {
            if (!cnt[temp[j].second]) ++have;
            ++cnt[temp[j].second];
            ++j;
        }
        if (have==k) {
            res = min(res, temp[j-1].first-temp[i].first+1);
        }
        if (cnt[temp[i].second]==1) --have;
        --cnt[temp[i].second];
    }
}

```

```

//更新其它数据
int lans = tr[u<<1].ans, rans = tr[u<<1|1].ans;
tr[u].ans = 0;
if (res!=N) tr[u].ans = res;
if (lans) tr[u].ans = min(tr[u].ans, lans);
if (rans) tr[u].ans = min(tr[u].ans, rans);
for (int i=1; i<=k; ++i) {
    if (tr[u<<1].lt[i]) tr[u].lt[i] = tr[u<<1].lt[i];
    else tr[u].lt[i] = tr[u<<1|1].lt[i];
    if (tr[u<<1|1].rt[i]) tr[u].rt[i] = tr[u<<1|1].rt[i];
    else tr[u].rt[i] = tr[u<<1].rt[i];
}
}

void build(int u, int l, int r) {
    tr[u] = {l, r};
    if (l==r) {
        tr[u].lt[a[l]] = tr[u].rt[a[r]] = 1;
        tr[u].lst = a[l];
        return ;
    }
    int mid = l+r>>1;
    build(u<<1, l, mid);
    build(u<<1|1, mid+1, r);
    push_up(u);
}

void update (int u, int idx, int v) {
    if (tr[u].l==tr[u].r) {
        tr[u].lt[tr[u].lst] = tr[u].rt[tr[u].lst] = 0;
        tr[u].lt[v] = tr[u].rt[v] = idx;
        tr[u].lst = v;
        return ;
    }
    int mid = tr[u].l+tr[u].r>>1;
    if (idx<=mid) update(u<<1, idx, v);
    if (idx>mid) update(u<<1|1, idx, v);
    push_up(u);
}

```



```

int main() {
    freopen("truth.in", "r", stdin);
    freopen("truth.out", "w", stdout);
    scanf("%d %d %d", &n, &k, &m);
    for (int i=1; i<=n; ++i) scanf("%d", &a[i]);
    build(1, 1, n);
    while (m--) {
        int op;
        scanf("%d", &op);
        if (op==1) {
            int idx, val;
            scanf("%d %d", &idx, &val);
            update(1, idx, val);
        }
        else printf("%d\n", tr[1].ans? tr[1].ans: -1);
    }
    return 0;
}

```

考察知识点

双指针 线段树 分块 区间问题

T4 异或区间 (xor)

$$a_i \oplus a_{i+1} \oplus \dots \oplus a_j \leq \max\{a_i, a_{i+1}, \dots, a_j\}$$

首先考虑简化的问题 $\max\{a_i, a_{i+1}, \dots, a_j\} = k$ 为定值, 那这个问题比较容易, 对数组求异或前缀和 s , 即求 $s_i \oplus s_j \leq k$ 的 $(i, j), i, j \in [0, n], i \neq j$

建立01字典树, 从0到n依次插入 s_i , 每次插入前查找字典树内与 s_i 异或值小于等于 k 的元素个数。

这里贴出插入和查找代码:

```

void insert(int x){
    int u=0;
    for(int i=30,o;~i;--i){
        o=(x>>i)&1;
        if(!trie[u][o]) trie[u][o]=++nNodes;
        u=trie[u][o];
        ++cnt[u];
    }
}

LL find(int x){
    LL sum=0;
    int u=0;
    for(int i=30,o1,o2;~i;--i){
        o1=(x>>i)&1,o2=(x>>i)&1;
        if(o1) u=trie[u][o2^1];
        else {sum+=cnt[trie[u][o2^1]];u=trie[u][o2];}
        if(!u) return sum;
    }
    sum+=cnt[u];
    return sum;
}

```

其中 $cnt(u)$ 表示字典树内以 u 为根的子树内有多少个元素，可认为是子树大小。

接下来考虑原问题，发现瓶颈在于 $\max\{a_i, a_{i+1}, \dots, a_j\}$ 不是个定值，可以发现在一段区间内其实最值不会发生变化。

因此可以考虑找到 a 的最大值 a_i 。那么凡是跨过 i 的区间的查找都有 $k = a_i$ 。接下来在 $[1, i - 1]$ 和 $[i + 1, n]$ 区间内进行分治。分别找到两个子区间内的最大值.....以此类推。

不断找最大值的过程，其实就是静态区间RMQ。用ST表可以做到 $O(n \log n)$ 预处理， $O(1)$ 查询。不过，这里的 RMQ 问题比较特殊，可以用笛卡尔树做到 $O(n)$ 预处理， $O(1)$ 查询。

```

a[0]=INF; st[top=1]=0;
for(int i=1,u;i<=n;++i)
{
    while(top&&a[st[top]]<=a[i]) --top; //找出递减的右链中第一个比a[i]大的数
    u=st[top];
    l[i]=r[u];
    r[u]=i; //修改右链
    st[++top]=i;
}

```

那么做法就很清楚了：构建笛卡尔树，然后在笛卡尔树上进行dfs. 访问到一个节点 u 时，首先处理其子树内的询问，并分别获得其左右子树内元素构成的字典树 TrL , TrR 。

接下来就是合并区间 $[l(u), u]$ 和 $[u + 1, r(u)]$ 。统计跨过 u 的询问时，可以选定其中一个区间。枚举其中的元素，在另一个区间对应的字典树内进行 `find` 询问，并对答案进行贡献。统计完毕后，我们还需得到 u 子树内所有元素组成的字典树。直接进行字典树合并是非常困难的，不妨直接将一个区间内的数直接全部插入另一区间对应的字典树（例如将 u 左侧的数全部插入 TrR ）。

容易发现，以上一次合并的复杂度为 $O(len \log s)$ ，其中 len 为选定区间的长度。那么，为了得到更小的时间开销，我们应该选择较短的那个区间作为选定区间。简而言之，枚举短区间的元素到长区间的字典树内查询，再将短区间的字典树并入长区间，即所谓的 **启发式合并**。这样，总的均摊复杂度下降为 $O(n \log n \log s)$ ，其中 $\log s \leq 31$ ，可以看作一个大常数。

全部代码：

```
/*
```

思路:首先对原序列构建笛卡尔树

在笛卡尔树上dfs, 每次访问到的结点为子树(区间 $[l, r]$)的最大值, 设下标为 i

考虑类似cdq分治的做法, 先分别统计 $[l, i-1]$ 和 $[i+1, r]$ 内的答案, 再统计跨过 i 的贡献

枚举长度较小的一段, 到长度较大的一侧的字典树中统计答案, 最后再进行“小并大”启发式合并即可

```
*/
```

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
typedef long long LL;
```

```
const int N=1e5+5;
```

```
const int INF=INT_MAX;
```

```
int a[N], s[N], n;
```

```
int l[N], r[N], st[N], top; //笛卡尔树
```

```
struct Node
```

```
{
```

```
    int son[2], val;
```

```
    LL cnt;
```

```
    Node(){son[0]=son[1]=0; cnt=0LL; val=-1;}
```

```
};
```

```
struct Trie
```

```
{
```

```
    int n;
```

```
    vector<Node> trie;
```

```
    Trie():n(0){trie.clear(); trie.push_back(Node());}
```

```
    void insert(int x)
```

```
    {
```

```
        int u=0;
```

```
        for(int i=30, o=~i; --i)
```

```
        {
```

```
            o=(x>>i)&1;
```

```
            if(!trie[u].son[o]){trie[u].son[o]=++n; trie.push_back(Node());}
```

```
            u=trie[u].son[o];
```

```
            ++trie[u].cnt;
```

```
        }
```

```
        trie[u].val=x;
```

```
    }
```

```
    LL find(int x, int k)
```

```
    {
```

```
        int u=0;
```

```

LL sum=0;
for(int i=30,o1,o2;~i;--i)
{
    o1=(k>>i)&1;
    o2=(x>>i)&1;
    if(o1){sum+=trie[trie[u].son[o2]].cnt; u=trie[u].son[o2^1];}
    else u=trie[u].son[o2];
    if(u==0) return sum;
}
sum+=trie[u].cnt;
return sum;
}
} A;
LL ans;
Trie dfs(int u,int L,int R) //统计笛卡尔树中以u为根的子树的答案
{
    if(!u || L>R) return Trie();
    if(R==L+1)
    {
        ++ans;
        Trie tree=Trie();
        tree.insert(s[L]);
        tree.insert(s[R]);
        return tree;
    }
    if(L==R)
    {
        Trie tree=Trie();
        tree.insert(s[L]);
        return tree;
    }
    Trie A=dfs(l[u],L,u-1);
    Trie B=dfs(r[u],u,R);
    int szL=u-1-L+1,szR=R-u+1;
    if(szL<szR) // 左并右
    {
        for(int i=L;i<=u-1;++i)
            ans+=B.find(s[i],a[u]);
        for(int i=L;i<=u-1;++i)

```

```

        B.insert(s[i]);
    return B;
}
else
{
    for(int i=u;i<=R;++i)
        ans+=A.find(s[i],a[u]);
    for(int i=u;i<=R;++i)
        A.insert(s[i]);
    return A;
}
return Trie();
}
int main()
{
    freopen("xor.in", "r", stdin);
    freopen("xor.out", "w", stdout);
    cin>>n;
    for(int i=1;i<=n;++i) cin>>a[i];
    for(int i=1;i<=n;++i) s[i]=s[i-1]^a[i];
    //构建笛卡尔树(大根堆)
    a[0]=INF; st[top=1]=0;
    for(int i=1,u;i<=n;++i)
    {
        while(top&&a[st[top]]<=a[i]) --top; //找出递减的右链中第一个比a[i]大的数
        u=st[top];
        l[i]=r[u];
        r[u]=i; //修改右链
        st[++top]=i;
    }
    A=dfs(r[0],0,n);
    cout << ans << "\n";
    return 0;
}

```

注意字典树如果直接开 10^5 大小的数组可能会 MLE，可以用 vector 代替。

考察知识点

二分 01字典树 笛卡尔树 启发式合并 字典树合并